

Penerapan *Particle Swarm Optimization* (PSO) untuk *Traveling Salesperson Problem*: Studi Kasus Kabupaten/Kota di Jawa Barat

Naufal Akmal Rizqulloh¹, Adhiya Radhin Fasya², Faiz Naufal Huda³, Insan Anshary Rasul⁴, Daffa Naufal Mumtaz⁵

Departemen Ilmu Komputer, Sekolah Sains Data, Matematika, dan Informatika
IPB University, Bogor, Indonesia

¹G6401231065 ²G6401231068 ³G6401231124 ⁴G6401231132 ⁵G6401231168

Mata Kuliah: Pengantar Kecerdasan Komputasional (KOM1326)

Abstract—This paper presents a hybrid Discrete Particle Swarm Optimization (PSO) combined with Greedy Nearest Neighbor seeding and 2-opt local search to solve the Traveling Salesperson Problem (TSP) on 27 real-world locations in West Java. The TSP is NP-hard; exact algorithms become intractable beyond 15 cities due to factorial state-space explosion. Our Memetic PSO seeds one particle with a Greedy route and periodically applies 2-opt sweeps to eliminate edge crossovers. An auto-tuning module selects optimal hyperparameters from four presets. The system is implemented as a modular Python pipeline with an interactive Streamlit dashboard and a Jupyter Notebook for academic analysis. Experiments show that the hybrid PSO reduces the Greedy baseline from 891.94 km to 788.08 km (11.64% improvement) in 9.79 s, with early stopping triggered by swarm convergence validated through particle diversity metrics.

Keywords—Particle Swarm Optimization, Traveling Salesperson Problem, 2-opt, Greedy Nearest Neighbor.

Abstrak—Makalah ini menyajikan hibridisasi Particle Swarm Optimization (PSO) Diskrit dengan penyemaian Greedy Nearest Neighbor dan pencarian lokal 2-opt untuk menyelesaikan Traveling Salesperson Problem (TSP) pada 27 lokasi riil di Jawa Barat. TSP merupakan masalah NP-hard di mana algoritma eksak tidak layak secara komputasi pada skala besar akibat ledakan ruang keadaan faktorial. PSO Memetik kami menyemai satu partikel dengan rute Greedy dan secara periodik menerapkan 2-opt untuk menghilangkan persilangan jalur. Modul auto-tuning memilih hyperparameter optimal dari empat preset. Sistem diimplementasikan sebagai pipeline Python modular dengan dashboard Streamlit interaktif dan Jupyter Notebook untuk analisis akademis. Hasil eksperimen menunjukkan PSO hibrida mengoptimalkan baseline Greedy dari 891.94 km menjadi 788.08 km (peningkatan 11.64%) dalam 9.79 s, dengan early stopping dipicu oleh konvergensi swarm yang divalidasi melalui metrik diversitas partikel.

Kata Kunci—Particle Swarm Optimization, Traveling Salesperson Problem, 2-opt, Greedy Nearest Neighbor.

I. PENDAHULUAN

Traveling Salesperson Problem (TSP) merupakan salah satu masalah optimasi kombinatorial tertua dan paling intensif dipelajari dalam ilmu komputer [4]. Tujuan TSP adalah menemukan rute siklik terpendek (*Hamiltonian cycle*) yang mengunjungi sekumpulan kota masing-masing tepat satu kali dan kembali ke kota asal. TSP diklasifikasikan sebagai masalah *NP-hard*, yang berarti tidak ada algoritma waktu polinomial yang diketahui dapat memecahkan seluruh instansi masalah secara eksak.

Untuk jumlah kota kecil ($N < 12$), algoritma eksak seperti *Branch and Bound* atau *A** dapat menemukan rute optimal mutlak. Namun, ketika jumlah kota bertambah ($N \geq 15$), ruang pencarian membengkak secara faktorial ($N!$). Sebagai contoh, untuk 27 lokasi di Jawa Barat, terdapat $27! \approx 1.08 \times 10^{28}$ kemungkinan rute, sehingga algoritma eksak mengalami kehabisan memori dan waktu komputasi yang tidak terbatas.

Oleh karena itu, algoritma metaheuristik seperti *Particle Swarm Optimization* (PSO) [1] menjadi solusi praktis. PSO merupakan bagian dari paradigma kecerdasan komputasional (*computational intelligence*) [5] yang terinspirasi dari perilaku kolektif kawanan burung. Algoritma heuristik *Greedy Nearest Neighbor* memberikan solusi cepat namun sering terjebak pada optimum lokal. Makalah ini mengintegrasikan PSO diskrit dengan penyemaian *Greedy* dan pencarian lokal 2-opt (*Memetic PSO*) untuk menyelesaikan TSP riil di Jawa Barat.

Tujuan penelitian ini adalah: (1) mengimplementasikan algoritma *Discrete Memetic PSO* berbasis *swap operator*, (2) membandingkan efisiensi rute antara *Greedy Nearest Neighbor* dan PSO Memetik, (3) membangun sistem interaktif berbasis *Streamlit* untuk visualisasi proses optimasi, dan (4) memvalidasi konvergensi algoritma menggunakan metrik diversitas partikel.

II. TINJAUAN PUSTAKA

A. Traveling Salesperson Problem (TSP)

Secara matematis, TSP direpresentasikan sebagai graf $G = (V, E)$ di mana V adalah himpunan kota dan E adalah himpunan jalur penghubung [4]. Jarak antara kota i dan j dilambangkan $d(i, j)$. Tujuan TSP adalah menemukan permutasi π yang meminimalkan total jarak rute siklik:

$$\min \sum_{i=1}^{N-1} d(\pi(i), \pi(i+1)) + d(\pi(N), \pi(1))$$

TSP termasuk kelas NP -hard berdasarkan teorema Cook–Karp, sehingga pendekatan metaheuristik diperlukan untuk instansi berskala besar.

B. Particle Swarm Optimization (PSO)

PSO diperkenalkan oleh Kennedy dan Eberhart [1], terinspirasi oleh perilaku sosial kawanan burung. Setiap partikel memperbarui kecepatan (V) dan posisi (X) berdasarkan memori pribadinya ($pbest$) dan memori kelompok ($gbest$). PSO termasuk dalam keluarga algoritma kecerdasan komputasional (*computational intelligence*) [5] yang tidak memerlukan informasi gradien dan mampu menjelajahi ruang pencarian secara global.

C. Discrete PSO (Permutation-Based)

Karena TSP memiliki ruang keadaan diskrit berupa permutasi, pembaruan vektor standar tidak dapat diterapkan. Model aljabar *swap operator* yang diperkenalkan Clerc [2] mengadaptasi PSO untuk domain permutasi, di mana posisi direpresentasikan sebagai urutan indeks kota dan kecepatan sebagai barisan operasi pertukaran.

D. Pencarian Lokal 2-Opt

Algoritma *2-opt* [3] merupakan teknik pencarian lokal untuk memperbaiki rute TSP dengan memilih dua sisi rute non-bersebelahan dan membalik segmen di antaranya. Proses diulang hingga tidak ada perbaikan, menghasilkan rute bebas persilangan jalur (*crossing edges*).

III. METODOLOGI PENELITIAN

A. Pengumpulan Data

Data diperoleh dari file CSV yang berisi nama, *latitude*, dan *longitude* 27 kabupaten/kota di Jawa Barat. Data dimuat dan dibersihkan menggunakan fungsi `load_and_filter_data()` yang memvalidasi struktur, menghapus kolom tidak relevan, dan menangani nilai kosong. Daftar koordinat ditampilkan pada Tabel I.

TABEL I
Daftar 27 Titik Koordinat Wilayah Jawa Barat

No.	Kabupaten / Kota	Latitude	Longitude
1	Bandung	-7.0635	107.6338

No.	Kabupaten / Kota	Latitude	Longitude
2	Bandung Barat	-6.9054	107.4124
3	Banjar	-7.3805	108.5610
4	Bekasi	-6.1979	107.1598
5	Bogor	-6.5454	107.0021
6	Ciamis	-7.4375	108.6452
7	Cianjur	-7.0538	107.0692
8	Cimahi	-6.8823	107.5445
9	Cirebon	-6.7614	108.4559
10	Depok	-6.3879	106.8161
11	Garut	-7.3429	107.7031
12	Indramayu	-6.4537	108.1853
13	Karawang	-6.2634	107.4294
14	Kota Bandung	-6.9033	107.6299
15	Kota Bekasi	-6.2849	106.9735
16	Kota Bogor	-6.5952	106.8000
17	Kota Cirebon	-6.7436	108.5562
18	Kota Sukabumi	-6.9357	106.9315
19	Kota Tasikmalaya	-7.3600	108.2279
20	Kuningan	-6.9876	108.5538
21	Majalengka	-6.8077	108.2836
22	Purwakarta	-6.5925	107.4036
23	Subang	-6.5007	107.7343
24	Sukabumi	-7.0782	106.7377
25	Sumedang	-6.8102	107.9617
26	Tasikmalaya	-7.4280	108.0691
27	Waduk Cirata	-6.7552	107.2857

B. Perhitungan Jarak Geodesik (Haversine)

Untuk rute geografis riil, metrik *Euclidean* tidak akurat karena kelengkungan bumi. Formula *Haversine* digunakan:

$$a = \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos(\varphi_1) \cdot \cos(\varphi_2) \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right)$$

$$c = 2 \cdot \arcsin(\sqrt{a})$$

$$d = R \cdot c$$

di mana $R \approx 6371$ km adalah radius rata-rata bumi, φ adalah *latitude*, dan λ adalah *longitude*.

C. Algoritma Greedy Nearest Neighbor

Mulai dari kota indeks 0, rute dibangun dengan memilih kota terdekat yang belum dikunjungi secara berulang:

$$v_{\text{next}} = \arg \min_{u \in \text{Unvisited}} d(v_{\text{current}}, u)$$

Algoritma ini memiliki kompleksitas $O(N^2)$ dan menjadi *baseline* pembandingan.

D. Aljabar Swap Operator untuk PSO Diskrit

Representasi diskrit PSO menggunakan aljabar *swap operator* [2]:

1. *Swap Operation* $SO(i, j)$: pertukaran elemen pada indeks i dan j dalam permutasi rute.
2. *Velocity* (V): barisan berurutan dari *swap operations*: $V = [SO(i_1, j_1), SO(i_2, j_2), \dots]$
3. Pengurangan posisi ($X_2 - X_1$): menghasilkan barisan *swap* yang mengubah X_1 menjadi X_2 .
4. Perkalian skalar ($p \cdot V$): mempertahankan setiap *swap* dengan probabilitas $p \in [0, 1]$.
5. Pembaruan kecepatan:

$$V_{t+1} = wV_t + c_1r_1(P_{\text{best}} - X_t) + c_2r_2(G_{\text{best}} - X_t)$$

di mana w adalah bobot inersia, c_1, c_2 adalah faktor kognitif dan sosial, r_1, r_2 adalah bilangan acak $[0, 1]$.

6. Pembaruan posisi: $X_{t+1} = X_t + V_{t+1}$.

E. Optimasi Hibrida & Memetic

Lima teknik kunci dirancang untuk meningkatkan kualitas solusi dan mencegah konvergensi prematur:

1. *Heuristic Seeding*: partikel ke-0 disemai dengan rute *Greedy*, menjamin pencarian dimulai dari *baseline* kuat, sementara partikel lainnya diinisialisasi acak untuk menjaga diversitas awal.
2. *Velocity Cap*: panjang barisan *swap* dibatasi hingga $\lfloor N/2 \rfloor$ operasi per iterasi, mencegah perubahan posisi yang terlalu ekstrem dan menstabilkan dinamika pencarian.
3. *Periodic 2-Opt*: setiap 5 iterasi, pencarian lokal *2-opt* [3] diterapkan pada 25% partikel *elite* (jarak terbaik saat itu) untuk menghilangkan persilangan jalur secara periodik:

$$\text{Jalur Baru} = [v_1, \dots, v_{i-1}, v_j, v_{j-1}, \dots, v_i, v_{j+1}, \dots]$$

4. *Adaptive Mutation Decay*: laju mutasi diturunkan secara linier terhadap kemajuan iterasi sehingga eksplorasi tinggi di awal berangsur bergeser menjadi eksploitasi di akhir:

$$\mu_t = \mu_0 \cdot (1 - 0.9 \cdot t/T)$$

di mana $\mu_0 = 0.05$, t adalah iterasi saat ini, dan T adalah batas iterasi maksimum.

5. *Diversity-Based Partial Restart*: jika koefisien variasi (*CoV*) jarak seluruh partikel turun di bawah ambang 1%, sebanyak 25% partikel dengan kinerja terburuk diinisialisasi ulang secara acak untuk memulihkan eksplorasi dan mencegah stagnasi prematur.

F. Auto-Tuning Parameter

Modul *auto-tuning* mengevaluasi empat *preset* konfigurasi melalui *grid search*, sebagaimana ditampilkan pada Tabel II. Setiap *preset* dievaluasi dengan 3 *trial* sesi PSO singkat (30 iterasi). *Preset* dengan rata-rata jarak akhir terendah dipilih sebagai konfigurasi optimal [4].

TABEL II

Preset Konfigurasi Parameter Auto-Tuning

<i>Preset</i>	w	c_1	c_2	Partikel
Balanced	0.7	1.5	1.5	20
Exploration-Heavy	0.9	2.0	1.0	30
Exploitation-Heavy	0.4	1.0	2.0	15
Lightweight	0.6	1.2	1.2	10

G. Kriteria Berhenti Awal (Early Stopping)

Jika jarak G_{best} tidak berkurang lebih dari 0.01 km selama K iterasi berturut-turut (*default* $K = 20$), *loop* dihentikan untuk menghemat waktu komputasi.

IV. IMPLEMENTASI SISTEM

Sistem dibangun dalam arsitektur modular terintegrasi yang terdiri dari tiga komponen utama [5]:

A. Mesin Perhitungan (*pso_engine.py*)

Modul komputasi dirancang dengan prinsip *Object-Oriented Programming* dan *type hints* penuh. Komponen utamanya: (1) fungsi *haversine_distance()* dan *euclidean_distance()* sebagai dua metrik jarak yang dapat dipilih; (2) fungsi aljabar *swap* *get_swap_sequence()*, *scale_velocity()* dengan klamping probabilitas ke $[0, 1]$, dan *apply_velocity()*; (3) kelas *GreedyTSPSolver* sebagai *baseline* $O(N^2)$; (4) kelas *Particle* yang merepresentasikan posisi rute, kecepatan *swap*, dan memori *pbest*; (5) kelas *PSOSolver* sebagai *orchestrator* yang mengintegrasikan kelima teknik optimasi hibrida; dan (6) fungsi *auto_tune_pso()* untuk seleksi *preset* otomatis berbasis *grid search*. Kebenaran seluruh komponen diverifikasi melalui 18 kasus uji pada *test_engine.py* dengan *pipeline* CI/CD berbasis *GitHub Actions*.

B. Dashboard Interaktif (*app.py*)

Antarmuka *Streamlit* dibangun modular dengan fungsi terpisah untuk tiap komponen visual. *Sidebar* menyediakan: unggah *dataset* CSV dengan deteksi kolom otomatis, pemilihan jumlah kota, metrik jarak, mode Auto-Tune/Manual, parameter $w, c_1, c_2, \text{mutation rate}, \text{early stopping}$, dan kontrol *seed* untuk reproduktibilitas. Panel utama menampilkan: peta interaktif *Plotly* pada *OpenStreetMap* dengan *toggle* rute *Greedy* dan PSO, *slider iteration playback* untuk menelusuri evolusi rute per iterasi, grafik konvergensi *real-time*, tabel perbandingan, dan tiga panel analisis performa akademik.

C. Notebook Akademis (*pso_tsp_demo.ipynb*)

Notebook Jupyter menyajikan analisis visual statis lengkap menggunakan *Matplotlib* dan *Seaborn* dalam tujuh seksi: (1) landasan teori aljabar *swap operator*; (2) pemuatan dan verifikasi data; (3) eksekusi *baseline Greedy*; (4) *auto-tuning* dan eksekusi PSO; (5) kurva konvergensi dan peta perbandingan rute; (6) tiga panel analisis kinerja (*bar chart* jarak, waktu

komputasi, kurva diversitas partikel); dan (7) analisis spasial berupa *heatmap* matriks jarak *pairwise* dan distribusi *edge hop length*.

V. HASIL DAN DISKUSI

Eksperimen dijalankan pada 27 lokasi Jawa Barat. Modul *auto-tuning* memilih *preset* Exploitation-Heavy dengan parameter $w = 0.4$, $c_1 = 1.0$, $c_2 = 2.0$, 15 partikel, *mutation rate* 0.05, batas 150 iterasi, dan *seed* 42 untuk reproduisibilitas.

A. Perbandingan Efisiensi Jarak Route

Rangkuman perbandingan disajikan pada Tabel III.

TABEL III
Perbandingan Performa Hasil Optimasi TSP

Metode	Jarak (km)	Selisih (km)	Efisiensi	Waktu
Jalur Acak	2701.19	+1809.25	-202.83%	N/A
Greedy NN	891.94	0.00 (Ref)	0.00%	2.00 ms
Memetic PSO	788.08	-103.86	11.64%	9.79 s

Memetic PSO berhasil memotong panjang rute sebesar 103.86 km (peningkatan efisiensi 11.64%) dibandingkan *baseline Greedy*. Peningkatan ini terjadi karena *Greedy* hanya mengambil keputusan lokal terbaik tanpa mempertimbangkan dampak jangka panjang rute, sehingga sering menyisakan kota-kota terjauh di akhir pencarian yang mengakibatkan lompatan rute balik sangat jauh. *Memetic PSO* mampu memataangkan rute secara global melalui kolaborasi antar partikel [1].

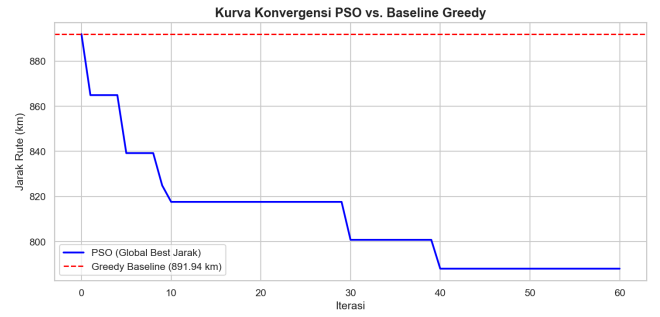
Perbandingan jarak rute antara ketiga metode (jalur acak, *Greedy NN*, dan PSO) ditampilkan secara visual pada Gambar 3 (panel kiri), yang secara jelas menunjukkan reduksi jarak bertahap dari pendekatan tanpa kecerdasan menuju optimasi PSO.

B. Analisis Waktu Komputasi

Greedy menyelesaikan pencarian dalam 2.00 ms karena kompleksitasnya $O(N^2)$, sedangkan *Memetic PSO* membutuhkan 9.79 s. Selisih waktu ini merupakan *trade-off* yang dapat diterima mengingat keuntungan penghematan rute 11.64%. Sebagian besar waktu PSO dihabiskan oleh operasi *2-opt* periodik setiap 5 iterasi; *Early Stopping* menghentikan *loop* pada iterasi ke-60 dari 150 iterasi maksimum. Perbandingan waktu komputasi kedua metode divisualisasikan pada Gambar 3 (panel tengah).

C. Analisis Konvergensi dan Diversitas Swarm

Kurva konvergensi pada Gambar 1 menunjukkan bahwa rute G_{best} langsung dimulai dari jarak *Greedy* (891.94 km) karena *heuristic seeding* pada partikel ke-0. Operasi *2-opt* periodik secara bertahap mengeliminasi persilangan jalur, sehingga jarak turun progresif hingga 788.08 km ketika *early stopping* terpicu pada iterasi ke-60.

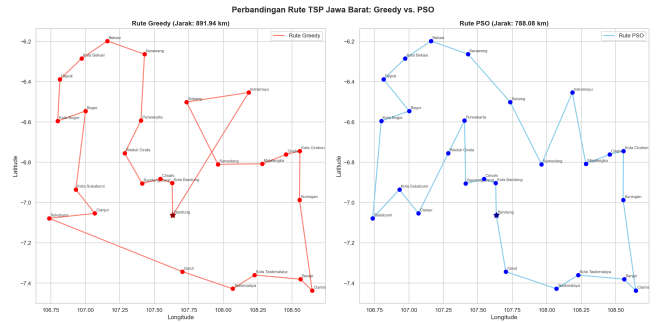


Gambar 1: Kurva konvergensi PSO vs. *baseline Greedy*.

Dari perspektif diversitas *swarm* (Gambar 3, panel kanan), standar deviasi *fitness* partikel dimulai tinggi (≈ 600 km) dan menyusut bertahap mendekati 0 setelah iterasi ke-60, menandakan konvergensi stabil [5].

D. Visualisasi Perbandingan Route

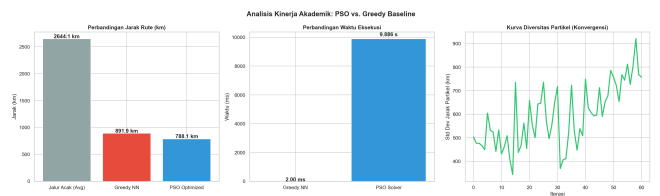
Gambar 2 menampilkan perbandingan visual rute *Greedy* (kiri) dan PSO (kanan) pada peta koordinat Jawa Barat. Terlihat bahwa rute *Greedy* memiliki beberapa persilangan jalur, sementara rute PSO lebih teratur dan menghindari lompatan jauh.



Gambar 2: Perbandingan rute TSP: *Greedy* (kiri, 891.94 km) vs. PSO (kanan, 788.08 km).

E. Analisis Performa Akademik

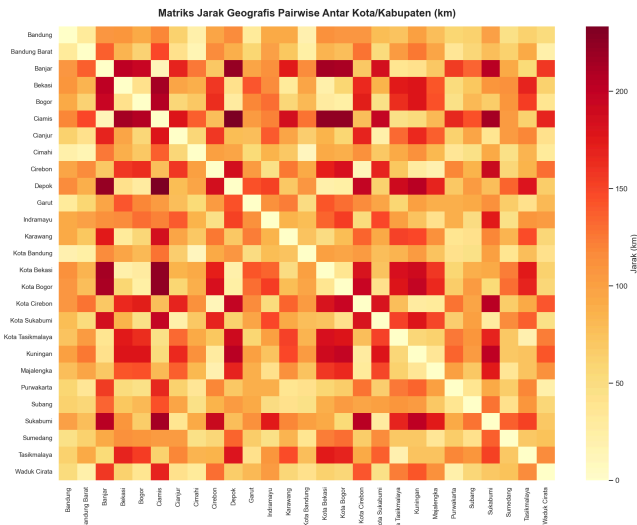
Gambar 3 menyajikan tiga panel analisis kinerja: (1) perbandingan jarak rute menunjukkan reduksi dari jalur acak (rata-rata 2701.2 km) ke *Greedy* (891.94 km) dan PSO (788.08 km); (2) perbandingan waktu komputasi menunjukkan *trade-off* antara kecepatan *Greedy* dan kualitas PSO; (3) kurva diversitas partikel memvalidasi konvergensi *swarm* secara matematis.



Gambar 3: Analisis kinerja akademik: perbandingan jarak, waktu, dan diversitas *swarm*.

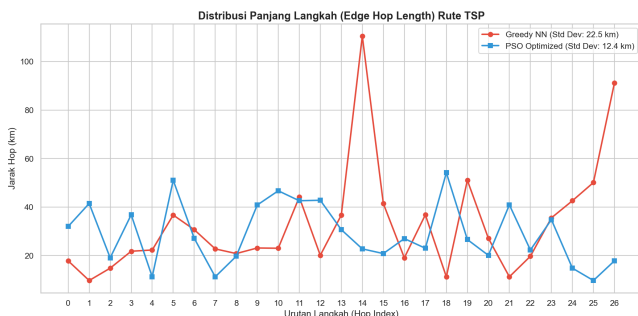
F. Analisis Spasial

Matriks jarak *pairwise* pada Gambar 4 mengonfirmasi adanya dua kluster padat: kluster metropolitan barat (Depok, Bogor, Bekasi) dan kluster Bandung Raya (Kota Bandung, Cimahi, Bandung Barat) dengan jarak < 20 km. Pola kluster ini menjelaskan mengapa rute optimal cenderung menyelesaikan kunjungan dalam satu kluster sebelum berpindah.



Gambar 4: Matriks jarak geodesik *pairwise* antar 27 kota/kabupaten (km).

Distribusi panjang langkah (*edge hop*) pada Gambar 5 menunjukkan bahwa *Greedy* NN memiliki lompatan sangat jauh (> 100 km) pada *hop* ke-14, sedangkan PSO memiliki distribusi lebih merata dengan standar deviasi lebih rendah (12.4 km vs. 22.5 km), membuktikan rute PSO lebih efisien [4].



Gambar 5: Distribusi panjang langkah (*edge hop length*) rute *Greedy* vs. PSO.

VI. KESIMPULAN

Penelitian ini membuktikan bahwa *Discrete PSO* yang dihibridisasi dengan *Greedy seeding* dan *2-opt* [3] merupakan solusi efektif untuk TSP riil. Metode hibrida ini menjamin rute selalu lebih optimal dari heuristik *Greedy* klasik, dengan peningkatan efisiensi 11.64% dan waktu komputasi 10 detik untuk 27 lokasi Jawa Barat.

Kontribusi utama meliputi: (1) implementasi PSO Diskrit berbasis *swap operator* [2] yang modular dengan *type hints* penuh dan 18 kasus uji terverifikasi; (2) lima teknik optimasi hibrida — *heuristic seeding*, *velocity cap*, *periodic 2-opt*, *memetic*, *adaptive mutation decay*, dan *diversity-based partial restart* — yang secara kolektif meningkatkan kualitas

solusi dan mencegah konvergensi prematur; (3) modul *auto-tuning* berbasis *grid search* empat *preset*; dan (4) sistem *dashboard* interaktif *Streamlit* dengan *iteration playback* dan analisis performa *real-time*.

Untuk penelitian selanjutnya, disarankan pengujian pada data dinamis menggunakan model *Haversine-Time*, perluasan jumlah kota ke skala nasional, serta eksplorasi algoritma metaheuristik lain seperti *Genetic Algorithm* [4] atau *Ant Colony Optimization* sebagai pembanding.

REFERENSI

- [1] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of ICNN'95*, vol. 4, pp. 1942-1948, 1995.
- [2] M. Clerc, "Discrete Particle Swarm Optimization, illustrated by the Traveling Salesman Problem," *New Optimization Techniques in Engineering*, pp. 219-239, 2004.
- [3] G. Croes, "A method for solving traveling-salesman problems," *Operations Research*, vol. 6, no. 6, pp. 791-812, 1958.
- [4] X.-S. Yang, *Nature-Inspired Metaheuristic Algorithms*, 2nd ed. Luniver Press, 2010.
- [5] A. P. Engelbrecht, *Computational Intelligence: An Introduction*, 2nd ed. Wiley, 2007.